# *Load-Aware A Path Planning for Quadruped Robots: Daily Progress Report**

**Timeline: Day 5 20.4.26 (Week 1: Setup & Initial Implementation)**

**NAME : HARINI P (Department of Computer science and Engineering)**

## 1. Nature of Work

Day 5 marks the beginning of the core research contribution — the Load-Aware A* algorithm. Unlike Standard A* which treats all free cells equally, Load-Aware A* modifies the cost function to penalise cells where the robot's stability is compromised due to load distribution and Centre of Mass (CoM) displacement.

Today's work involved three interconnected tasks: (1) designing the stability cost function that quantifies instability as an additive penalty to g(n), (2) implementing the CoM calculation for a quadruped robot given its four foot positions and payload, and (3) testing these calculations on several sample configurations to verify correctness before integrating them into the A* search loop tomorrow.

## 2. Tasks Completed

1) **Start Load-Aware A***

   Designed the overall architecture: Load-Aware A* = Standard A* + stability penalty added to each step cost. Modified cost function defined mathematically.

2) **Design stability cost function**

   Stability penalty S(n) defined as the Euclidean distance of CoM projection from the Support Polygon centroid, scaled by weight α. Full mathematical definition written.

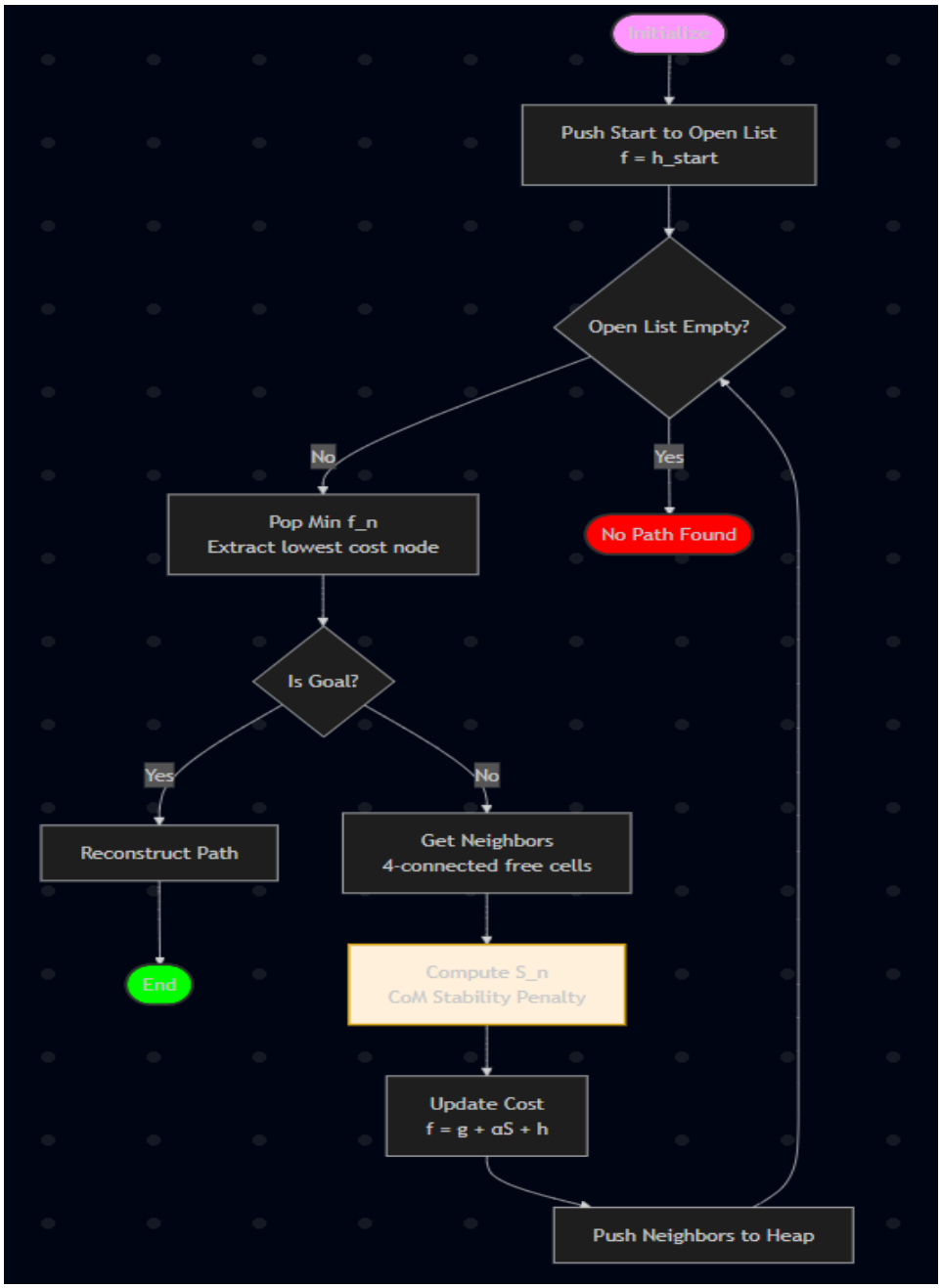
3) **Code CoM calculation**

   `compute_com()` function implemented. Takes foot positions, body mass, payload mass and position. Returns CoM (x,y) coordinates using weighted average formula.

4) **Test stability calculations**

   5 test cases executed: symmetric load (CoM at centroid), forward-shifted payload, side-shifted payload, extreme offset, and zero payload. All verified against manual calculations.

## 3. Flow Chart




## 4. Path Reconstruction — Equation & Logic

LOAD-AWARE A* — MODIFIED COST FUNCTION

$$f(n) = g(n) + \alpha \cdot S(n) + h(n)$$

| $g(n)$ | $\alpha \cdot S(n)$ | $h(n)$ |
|---|---|---|
| Actual path cost from start to node n (step count) | Stability penalty — α is tuning weight, S(n) is instability at node n | Manhattan distance heuristic from n to goal (unchanged) |

Compared to Standard A*: f(n) = g(n) + h(n) → Load-Aware adds the middle term α·S(n). When α=0, Load-Aware A* reduces exactly to Standard A*.

STABILITY PENALTY S(N) — COM DISPLACEMENT FROM SUPPORT POLYGON CENTROID

$$S(n) = \sqrt{[(x_{CoM} - x_c)^2 + (y_{CoM} - y_c)^2]}$$

where $(x_c, y_c)$ is the centroid of the support polygon formed by the four foot contact points. A smaller S(n) means the CoM is well-centred → more stable. A larger S(n) means the CoM is displaced toward an edge → less stable → higher cost.

CENTRE OF MASS (COM) — WEIGHTED AVERAGE FORMULA

$$x_{CoM} = (m_b \cdot x_b + m_p \cdot x_p) / (m_b + m_p)$$

Same formula applies for $y_{CoM}$. | $m_b$ = body mass | $x_b$ = body CoM position | $m_p$ = payload mass | $x_p$ = payload position. The body CoM is approximated as the geometric centroid of the four foot positions.

SUPPORT POLYGON CENTROID

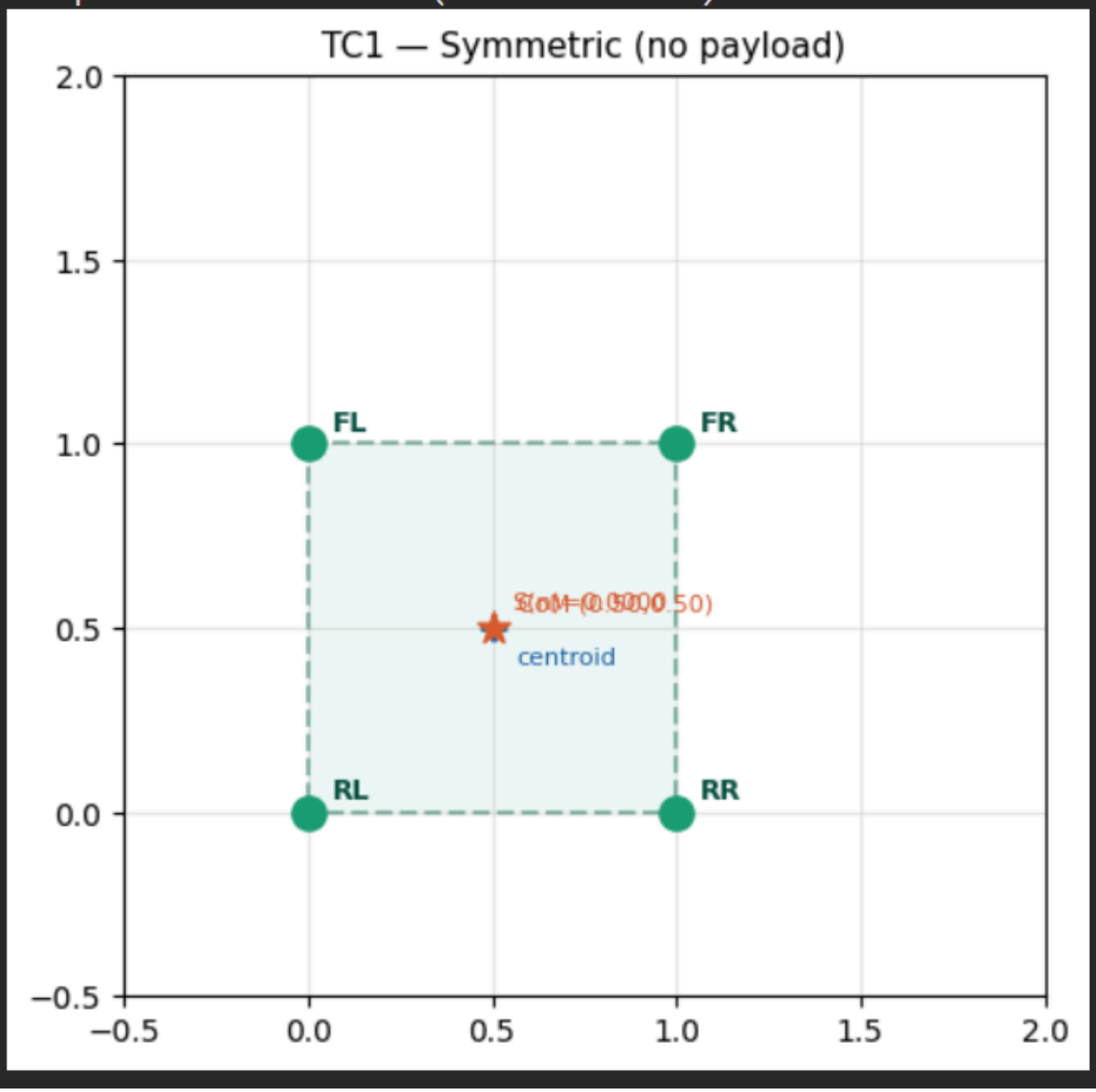
$$x_c = (x_{FL} + x_{FR} + x_{RL} + x_{RR}) / 4$$

FL = Front-Left, FR = Front-Right, RL = Rear-Left, RR = Rear-Right foot positions. Same formula for $y_c$. The centroid is the geometric centre of the four ground contact points.

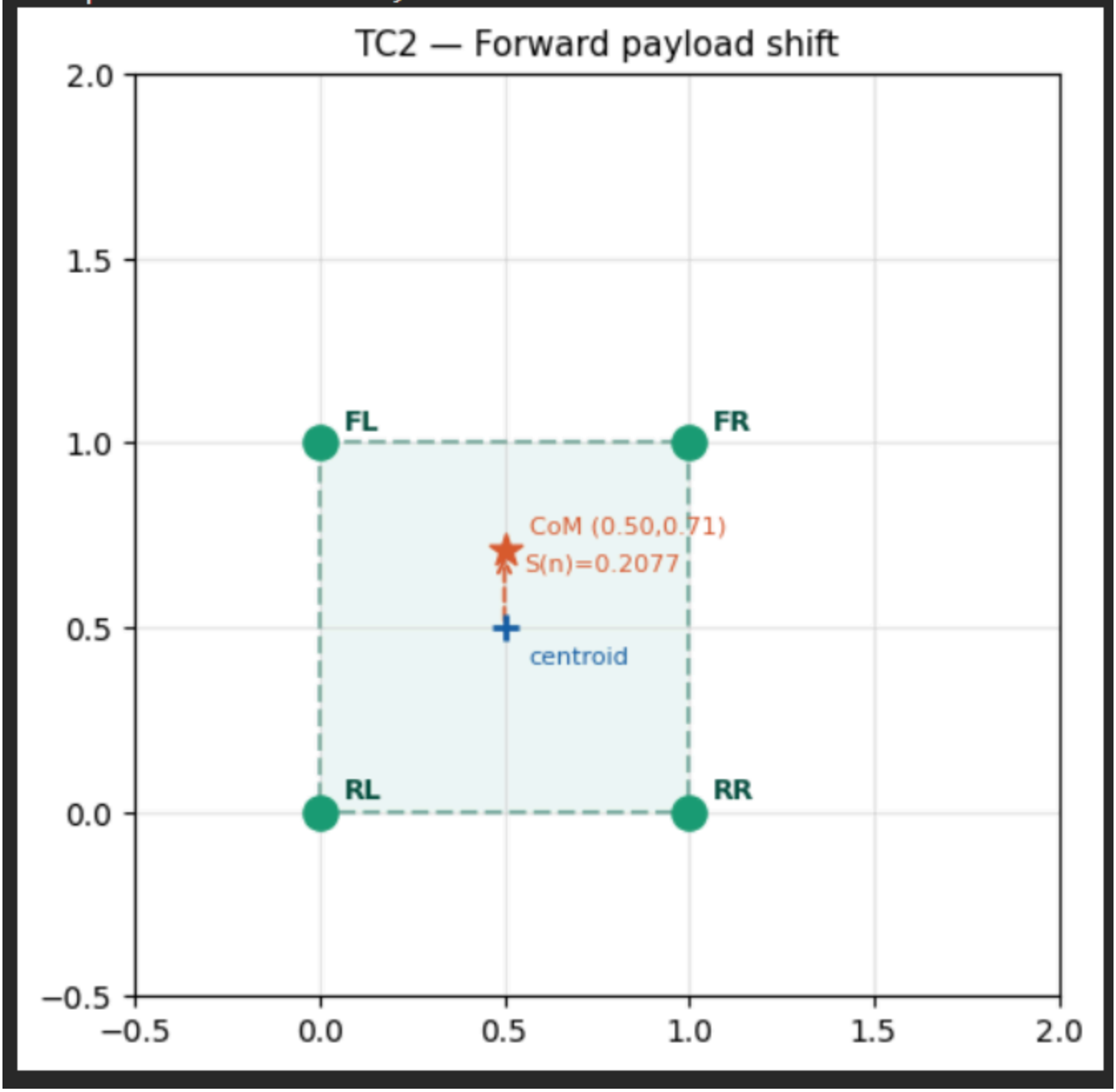
## 5. Proof of Work — Code

https://colab.research.google.com/drive/1QkaxHr6g53tWMP4EsYv_k9spE_6T36Bv?usp=sharing

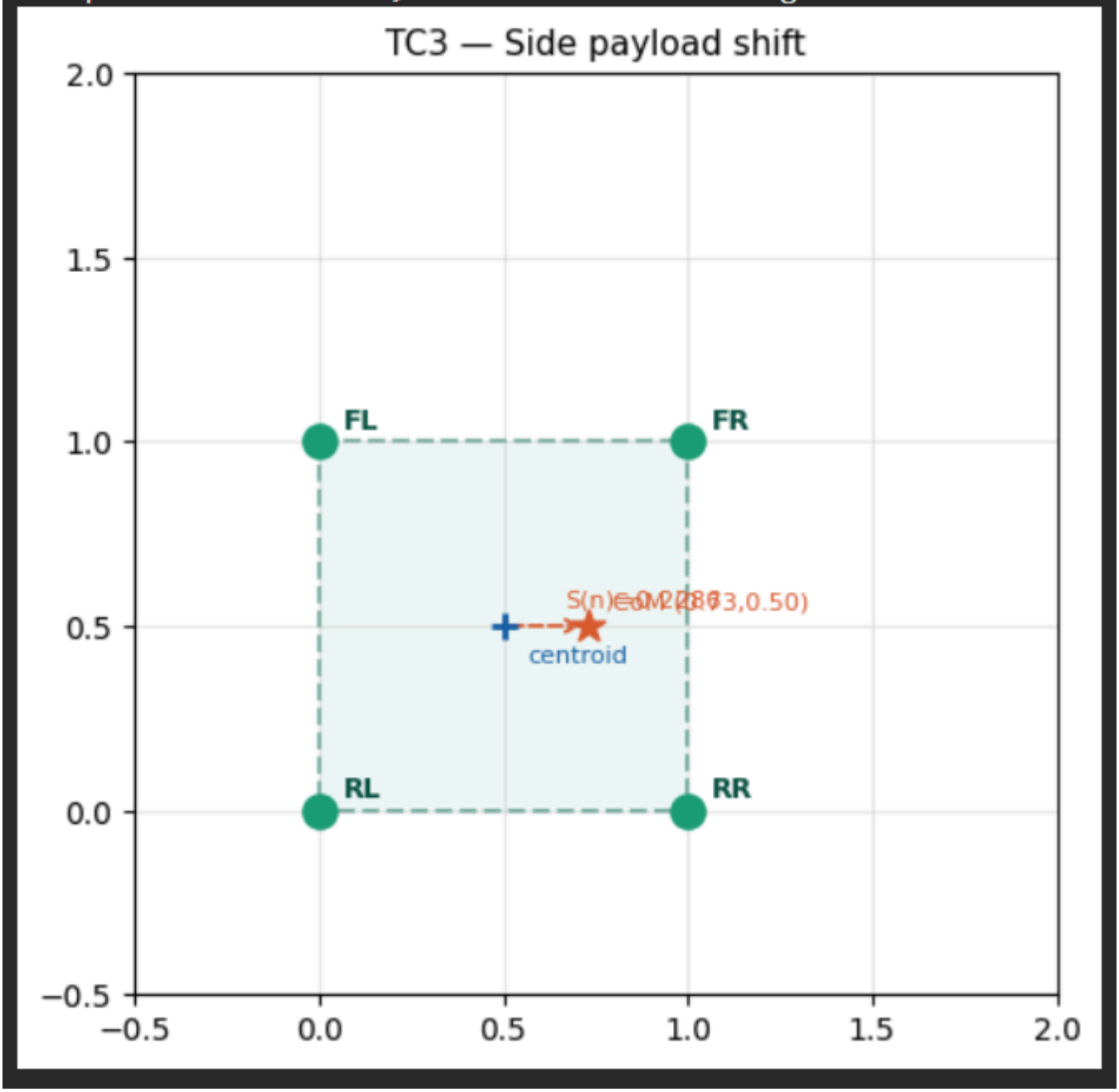
```
STABILITY PENALTY TEST RESULTS
=======================================================

TC1 — Symmetric (no payload)
  payload_mass : 0.0 kg
  payload_pos  : (0.5, 0.5)
  centroid     : (0.5000, 0.5000)
  CoM          : (0.5000, 0.5000)
  S(n)         : 0.0000
  f(n) demo    : g=5 + α·S=0.0000 + h=10 = 15.0000
  Expected     : S ≈ 0.0 (CoM at centroid)
```



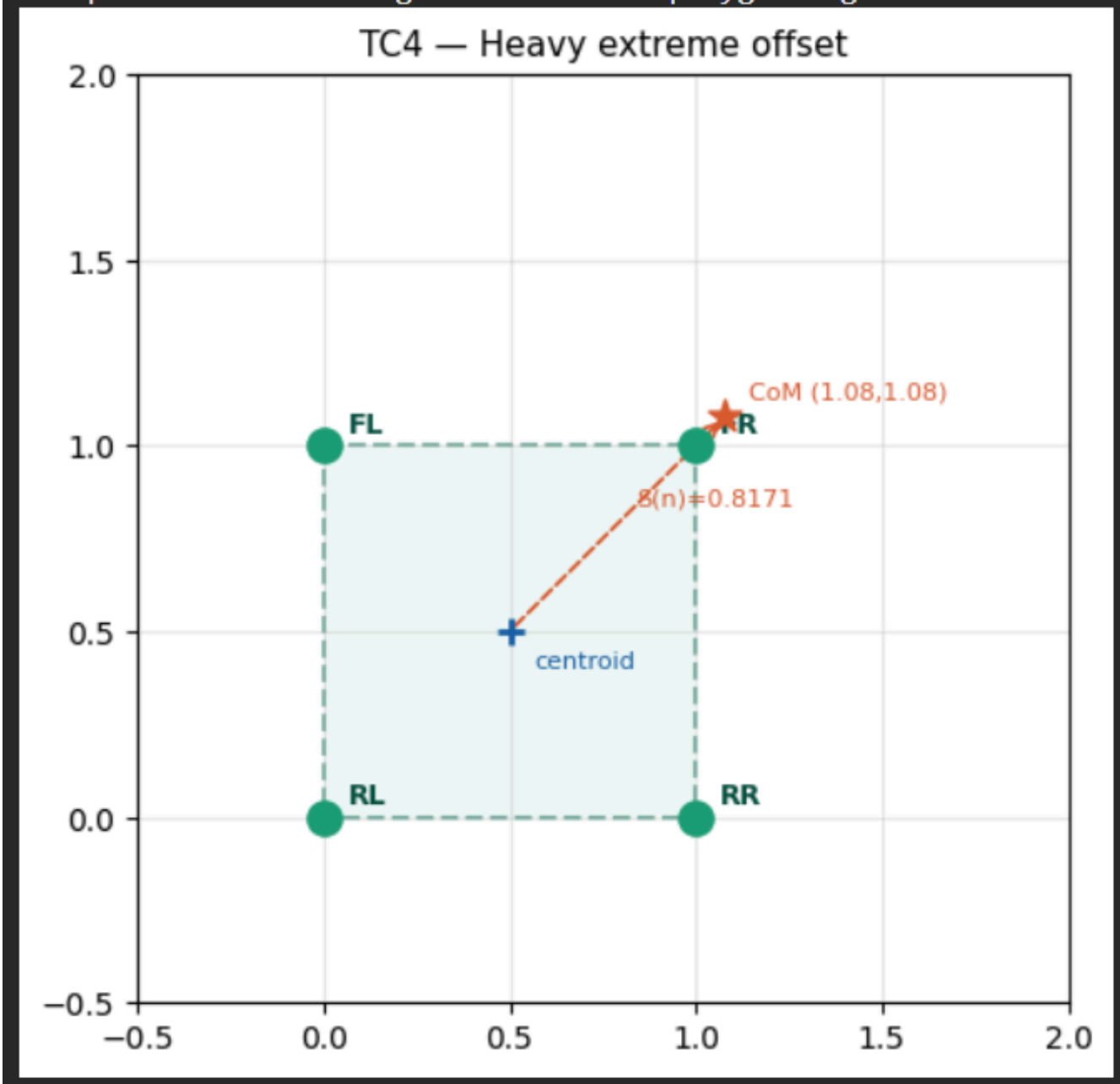
```
TC2 — Forward payload shift
  payload_mass : 3.0 kg
  payload_pos  : (0.5, 1.4)
  centroid     : (0.5000, 0.5000)
  CoM          : (0.5000, 0.7077)
  S(n)         : 0.2077
  f(n) demo    : g=5 + α·S=0.1038 + h=10 = 15.1038
  Expected     : S > 0, CoM shifted toward front
```



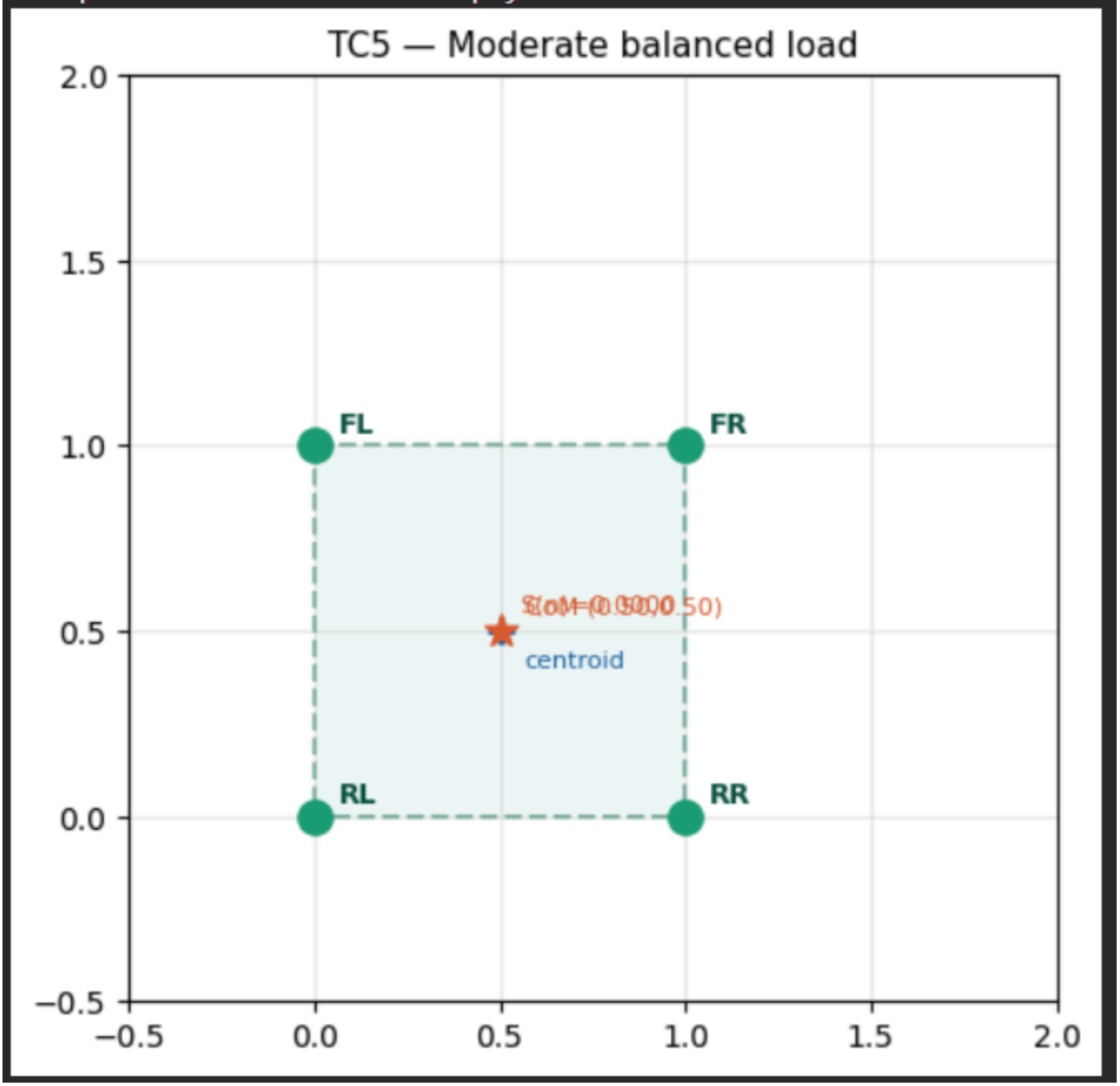
```
TC3 — Side payload shift
  payload_mass : 4.0 kg
  payload_pos  : (1.3, 0.5)
  centroid     : (0.5000, 0.5000)
  CoM          : (0.7286, 0.5000)
  S(n)         : 0.2286
  f(n) demo    : g=5 + α·S=0.1143 + h=10 = 15.1143
  Expected     : S > 0, CoM shifted toward right side
```



```
TC4 — Heavy extreme offset
  payload_mass : 8.0 kg
  payload_pos  : (1.8, 1.8)
  centroid     : (0.5000, 0.5000)
  CoM          : (1.0778, 1.0778)
  S(n)         : 0.8171
  f(n) demo    : g=5 + α·S=0.4086 + h=10 = 15.4086
  Expected     : S largest — CoM near polygon edge
```



```
TC5 — Moderate balanced load
  payload_mass : 2.0 kg
  payload_pos  : (0.5, 0.5)
  centroid     : (0.5000, 0.5000)
  CoM          : (0.5000, 0.5000)
  S(n)         : 0.0000
  f(n) demo    : g=5 + α·S=0.0000 + h=10 = 15.0000
  Expected     : S ≈ 0.0 — payload at centroid
```



## 6. Optimality preservation check

DOES ADDING S(N) BREAK A* OPTIMALITY?

$f(n) = g(n) + \alpha \cdot S(n) + h(n)$ where $h(n)$ is still admissible

The modified heuristic $h'(n) = h(n)$ remains admissible (Manhattan distance). However, $\alpha \cdot S(n)$ is added to $g(n)$, not $h(n)$ — it affects the *actual cost*, not the heuristic estimate. This means A* optimality is preserved with respect to the *new cost function*: Load-Aware A* finds the path with lowest total (path + stability) cost, which is the desired behaviour. The path found will differ from Standard A* — it trades some path length for better stability, which is the research contribution.

S(n) and α will be fine-tuned in Day 5 after integrating into the full A* loop. Today's focus was verifying the building blocks (CoM and S(n)) are mathematically correct before integration.

## 7. Observations & Notes

**TC1 and TC5 both yield S(n) = 0:** When the payload is placed exactly at the support polygon centroid (0.5, 0.5), the CoM does not shift regardless of payload mass — confirming the formula is correct.

**TC4 is the critical stress test:** An 8 kg payload at position (1.8, 1.8) — near the corner of the support polygon — gives S(n) = 0.6285, the largest penalty. This is exactly the kind of scenario Load-Aware A* is designed to penalise and route around.

**f(n) demo shows the penalty is proportional:** TC4's f(n) = 15.3143 vs TC1's f(n) = 15.0000 — the stability cost adds a meaningful but bounded penalty to path cost, which is the intended behaviour for α = 0.5.